

Faculty of Engineering & Computer Science
CSE211 – Software Engineering



HOSPITAL MANAGEMENT SYSTEM

Report 2: Back-End & Database Implementation



TEAM MEMBERS

1. Mostafa Kassab
2. Yassin Ali
3. Ali Hassan
4. Carol Joseph
5. Hana Hany
6. Norhan Mostafa

Submission Date: May 6, 2026



TABLE OF CONTENTS

Hospital Management System – Back-End & Database Implementation

1	Introduction & Project Overview	3
2	System Architecture	3
2.1	Architectural Overview	3
2.2	Request Flow	3
3	Technologies Used	4
4	Database Design	4
4.1	Collections Overview	4
4.2	Relationships & Indexing	5
5	API Design & Endpoints	5
5.1	Authentication Routes /api/auth	5
5.2	Patient Routes /api/patient	5
5.3	Doctor Routes /api/doctor	6
6	Core Functionalities & CRUD Operations	6
7	Security & Authentication	6
8	Development Workflow	7
8.1	Development Stages	7
8.2	Team Collaboration	7
9	Challenges & Solutions	7
10	Conclusion	8

1. INTRODUCTION & PROJECT OVERVIEW

The Hospital Management System is a web-based platform designed to streamline interaction between patients and healthcare providers. The system provides dedicated portals for two primary user roles — patients and doctors — each with tailored functionality supporting their respective workflows.

This report covers the back-end implementation phase of the project. The focus was on establishing a functional server-side infrastructure, connecting the system to a persistent database, and exposing a set of well-defined APIs for the front-end to consume. The goal was to build a reliable, secure, and maintainable foundation that supports all core system operations.

The back-end acts as the central hub of the application, responsible for data management, business logic, and secure communication between the client and the database. All interactions — from user authentication to appointment management and medical record handling — flow through this layer.

2. SYSTEM ARCHITECTURE

The system follows a standard three-tier client-server architecture, separating the presentation, application, and data layers so each can be developed, tested, and maintained independently.

2.1 Architectural Overview

Client Layer (Front-End)	A React-based single-page application communicating with the back-end through HTTP API requests, handling all UI rendering.
Application Layer (Back-End)	A Node.js server built with Express. Receives requests, applies logic, interacts with the database, and returns structured JSON responses.
Data Layer (Database)	A MongoDB database persisting all application data, accessed through Mongoose as an Object Data Modeling (ODM) library.

2.2 Request Flow

Every client request passes through a defined pipeline: received by the Express server → authenticated via middleware → routed to the appropriate controller → database query executed → structured response returned. Centralized error-handling middleware intercepts failures and returns consistent error messages without exposing internal details.

3. TECHNOLOGIES USED

CATEGORY	TECHNOLOGY	PURPOSE
Runtime Environment	Node.js	Server-side JavaScript execution
Web Framework	Express.js	HTTP routing and middleware management
Database	MongoDB	NoSQL persistent data storage
ODM Library	Mongoose	Schema definition and DB interaction
Authentication	JSON Web Tokens (JWT)	Stateless session management

CATEGORY	TECHNOLOGY	PURPOSE
Password Security	bcryptjs	Password hashing and verification
File Handling	Multer	File uploads (medical documents)
Environment Config	dotenv	Secure management of environment vars
Cross-Origin Support	CORS	Controlled cross-origin API access
Development Tool	Nodemon	Auto-restart server during development

4. DATABASE DESIGN

The database is built on MongoDB, a document-oriented NoSQL database suited for the flexible and nested data structures found in healthcare applications. Collections are defined using Mongoose schemas, which enforce data types, required fields, default values, and relationships between documents.

4.1 Collections Overview

COLLECTION	KEY FIELDS & PURPOSE
Users	Unified collection for patients, doctors, and admins differentiated by a role field. Stores full name, email, hashed password, phone, and profile image. Doctors additionally store specialization, department, and working schedule; patients store relevant medical history.
Appointments	Records all appointments linking patients to doctors. Stores date/time, specialty, reason, duration, assigned room, and a lifecycle status field. Cancellation details are captured when applicable.
Medical Records	Clinical records created by doctors post-visit. Captures diagnosis, symptoms, medications, lab results, and uploaded files. An amendment mechanism logs corrections without altering the original record.
Time Slots	Supports scheduling by maintaining available booking windows within a doctor's working hours, enabling structured and conflict-free appointment booking.

4.2 Relationships & Indexing

Collections are related through MongoDB ObjectId references resolved at query time using Mongoose's population feature. Database indexes are applied on frequently queried fields — such as doctor ID and time range for appointments, and patient ID and visit date for records — to ensure efficient retrieval as the dataset grows.

5. API DESIGN & ENDPOINTS

The back-end exposes a RESTful API organized into three route groups. All protected routes require a valid JWT token. Role-based access control ensures each user type accesses only their permitted endpoints.

5.1 Authentication Routes /api/auth

METHOD	ENDPOINT	DESCRIPTION
POST	/signup	Register a new user
POST	/login	Authenticate and receive a token

METHOD	ENDPOINT	DESCRIPTION
POST	/patient/signup	Register a patient account
POST	/patient/login	Patient login
POST	/doctor/signup	Register a doctor account
POST	/doctor/login	Doctor login
GET	/me	Get the authenticated user's profile
PATCH	/me	Update the authenticated user's profile

5.2 Patient Routes /api/patient

METHOD	ENDPOINT	DESCRIPTION
GET	/dashboard	Patient dashboard summary
GET	/doctors	Browse available doctors
GET	/time-slots	View available booking slots
GET	/appointments	List patient's appointments
POST	/appointments	Book a new appointment
PATCH	/appointments/:id/cancel	Cancel an appointment
GET	/records	View patient's medical records

5.3 Doctor Routes /api/doctor

METHOD	ENDPOINT	DESCRIPTION
GET	/dashboard	Doctor dashboard summary
GET	/schedule	View appointment schedule
PATCH	/schedule/day-availability	Update availability for a day
PATCH	/schedule/day-unavailable	Mark a day as unavailable
PATCH	/settings/working-hours	Update weekly working hours
PATCH	/appointments/:id/status	Update appointment status
GET	/patients	List patients seen by the doctor
GET	/patients/:id	View a specific patient's details
POST	/patients/:id/records	Create a medical record
PATCH	/records/:id/amend	Amend an existing record

6. CORE FUNCTIONALITIES & CRUD OPERATIONS

The system implements full CRUD operations across its main entities. The following summarizes the key functional areas covered during this phase.

FEATURE AREA	IMPLEMENTATION DETAILS
6.1 User Management	Users register through role-specific endpoints. Profiles can be retrieved and updated via authenticated routes. Passwords are always stored in hashed form and are never returned in any API response.
6.2 Appointment Management	Patients book appointments by selecting a time slot. Appointments move through a defined status lifecycle managed by the doctor. Both roles can trigger permitted status changes; cancellation details are recorded.
6.3 Medical Records	Doctors create clinical records after visits, capturing diagnoses, symptoms, medications, lab results, and files. An amendment mechanism logs corrections transparently without altering the original data.
6.4 Schedule Management	Doctors define and manage their weekly availability. This information feeds directly into the patient-facing booking flow, ensuring only valid and available time slots are shown.
6.5 File Uploads	The system supports uploading medical documents such as lab results and prescriptions. Files are stored server-side and their access URLs are saved within the relevant medical record.

7. SECURITY & AUTHENTICATION

SECURITY LAYER	IMPLEMENTATION
Authentication	JSON Web Tokens (JWT) for stateless authentication. Upon successful login, a signed token is issued to the client and included in the header of every subsequent protected request.
Authorization	Role-based access control enforced at the route level. Doctor and patient routes are independently protected, ensuring no cross-role data access is possible.
Password Security	Passwords are hashed before storage using bcryptjs and excluded from all query results by default, preventing accidental exposure in API responses.
Error Handling	Centralized error-handling middleware returns consistent responses without exposing internal system details to clients.
CORS	Configured to accept requests only from trusted origins, preventing unauthorized external access to the API.

8. DEVELOPMENT WORKFLOW

The team adopted an incremental development approach, building and validating the back-end layer by layer before integrating with the front-end. This minimized integration issues and kept the process organized.

8.1 Development Stages

#	STAGE	DESCRIPTION
1	Environment Setup	Server, environment configuration, and database connection established as the foundational layer.
2	Data Modeling	Database schemas designed and agreed upon before any controller logic was written, ensuring a stable data contract.

#	STAGE	DESCRIPTION
3	Authentication Layer	User registration, login, and token verification implemented and tested first, as prerequisites for all other features.
4	Feature Implementation	Each feature built as a self-contained unit — route, controller, and database interaction — allowing independent testing.
5	Middleware Integration	Authentication, authorization, file upload handling, and error management middleware integrated throughout.
6	Front-End Integration	React front-end connected and tested end-to-end once API endpoints were stable; mismatches resolved collaboratively.

8.2 Team Collaboration

Tasks were distributed among team members based on feature areas. Regular sync sessions aligned the team on API contracts and resolved blockers, with shared version control used to manage contributions.

9. CHALLENGES & SOLUTIONS

CHALLENGE	SOLUTION
Unified User Model	A single Users collection with a role field was adopted, applying conditional field requirements per role — keeping the schema clean without unnecessary collection duplication.
Appointment Lifecycle	A consistent status model was established defining valid transitions at each stage, applied uniformly across all appointment-related operations.
Record Integrity	An amendment logging mechanism was implemented that preserves the original record while maintaining a transparent and auditable history of all corrections.
Cross-Role Data Isolation	Role-based authorization was applied at the route level, and data queries were scoped to the authenticated user's identity, preventing cross-role data access.

10. CONCLUSION

The back-end implementation phase has produced a functional, secure, and well-structured server-side system for the Hospital Management System. The system covers all core requirements of Phase 2, including:

- ▶ User authentication and role-based access control
- ▶ Appointment management with a defined lifecycle model
- ▶ Medical record creation, retrieval, and amendment
- ▶ Schedule management and dynamic time-slot generation
- ▶ File upload support for clinical documents

The architecture is designed to be modular and extensible, allowing the team to continue refining and expanding the system as development progresses toward the final submission. The database design, API structure, and security measures collectively provide a solid and stable foundation for all remaining phases.